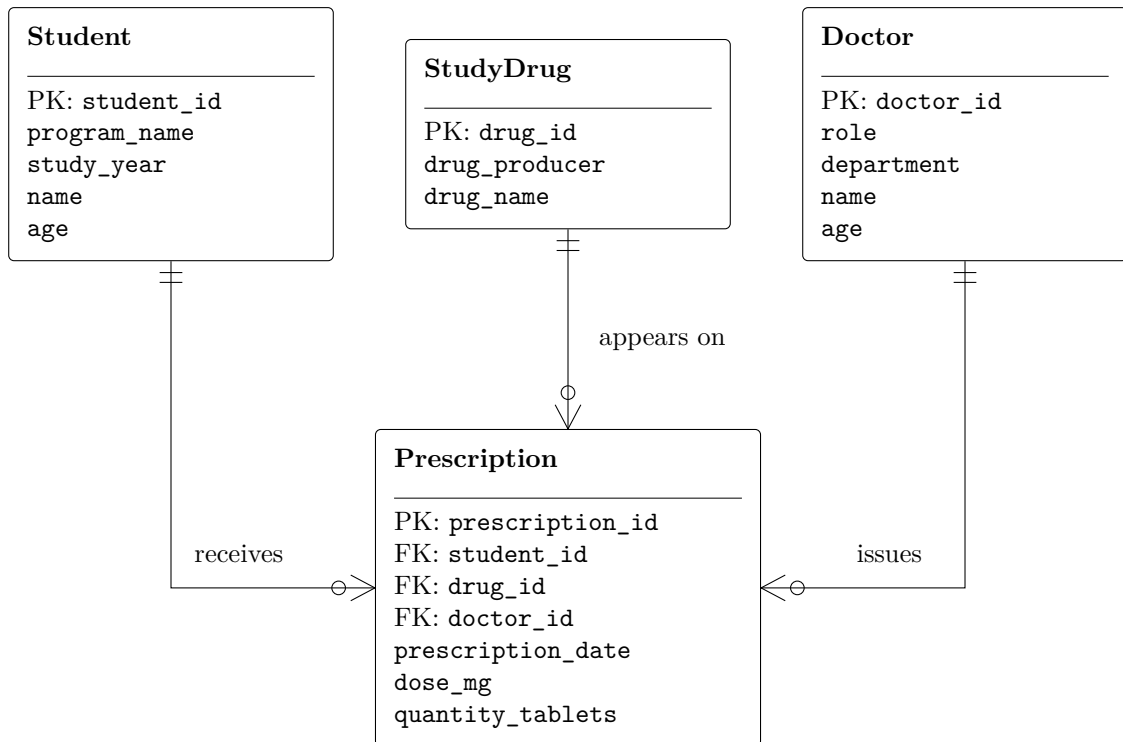


Student Prescription Database

Final entity relationship diagram



Reading the diagram: two bars mean exactly one, a circle with a crow's foot means zero or many. Each prescription belongs to exactly one student, one doctor and one drug. Each student, doctor or drug can be linked to zero or many prescriptions. PK means primary key. FK means foreign key.

Report outputs: StudentReport and DoctorReport are calculated using queries or views, rather than stored as extra tables. They can show prescription counts and summaries by drug. We do not average doses across different drugs because the resulting number would be hard to interpret.

1 Scope and design choices

Our societal problem is the non-medical use of study drugs among university students. For this database, we focus on recording prescriptions and describing prescription patterns. We want to answer which drugs students receive, which doctors issue prescriptions, how many prescriptions a student has, and which prescription is their most recent. We can also compare prescription patterns between study programmes or years.

A prescription record does not prove misuse. Our database does not record diagnoses, actual consumption, motivation or medicines obtained without a prescription. It therefore cannot tell us whether a prescription is medically appropriate or whether a student uses medication to improve academic performance.

1.1 How the data interacts

- A student can receive several prescriptions, but each prescription belongs to one student.
- A doctor can issue several prescriptions, but each prescription is issued by one doctor.
- A drug can appear on several prescriptions, but each prescription row records one drug.
- Students, doctors and drugs can be registered before any prescriptions exist.

Students and drugs have a many-to-many relationship over time. Prescription connects them and also records the doctor and event details. We use a separate prescription ID because the same student can receive the same drug more than once. A key containing only student ID and drug ID would not distinguish these events.

1.2 Entities, attributes and assumptions

- **Student:** student ID, name, age, programme name and study year. We store one current programme and study year per student.
- **Doctor:** doctor ID, name, age, role and department. We store one current role and department per doctor.
- **StudyDrug:** drug ID, drug name and producer. Each row describes one drug product, with one producer. The name–producer pair is assumed to identify the product uniquely.
- **Prescription:** prescription ID, student ID, doctor ID, drug ID, date, prescribed dose in milligrams and tablet quantity.

We keep **age** to match the ERD. It represents age when the record is entered or updated, so it needs maintenance as time passes. Storing date of birth instead could improve a future version, but we do not mix the two designs in this report. The IDs and drug names in the following examples are fictional and are not dosage advice. Example IDs are written as labels for readability; an implementation can use integer IDs.

2 Why we normalize the database

Normalization helps us avoid storing the same fact in several places. Otherwise, changing a student's name might require several updates, adding a doctor might require inventing a prescription, or deleting the last prescription might remove useful drug information. We apply 1NF, 2NF and 3NF in order. The tables below show the relevant columns at each stage, omitted event fields stay with their prescription throughout the process.

3 1NF

In the initial design, one student row contains lists of prescription numbers, drug IDs and dates. The highlighted cells hold several values and violate 1NF.

Before 1NF

Student ID	Name	Prescription no.	Drug ID	Date
S01	Alex	1, 2	D01, D02	2026-09-01, 2026-09-03
S02	Sam	1	D01	2026-09-04
S03	Jamie	1	D02	2026-09-05

After 1NF

Student ID	Name	Prescription no.	Drug ID	Date
S01	Alex	1	D01	2026-09-01
S01	Alex	2	D02	2026-09-03
S02	Sam	1	D01	2026-09-04
S03	Jamie	1	D02	2026-09-05

Each prescription now becomes a separate row. The matching prescription number, drug and date stay together. Each cell contains one value, so we can search and count individual prescriptions. We assume prescription numbers restart for each student. Therefore, the temporary primary key is **(student ID, prescription number)**: both values are needed to identify a row.

Student details still repeat, but repetition alone does not mean 1NF is violated. The next step checks whether each non-key attribute depends on the whole key.

4 2NF

Our table is now in 1NF, but student name and programme depend only on student ID. They do not depend on prescription number. This is a partial dependency on the composite key.

Before 2NF

Student ID (PK)	Prescription no. (PK)	Name	Programme	Drug ID
S01	1	Alex	Psychology	D01
S01	2	Alex	Psychology	D02
S02	1	Sam	Economics	D01
S03	1	Jamie	Biology	D02

Dependency causing the violation:

student ID $\rightarrow$ name, programme, study year, age.

After 2NF: Student

Student ID (PK)	Name	Programme	Study year	Age
S01	Alex	Psychology	2	23
S02	Sam	Economics	3	24
S03	Jamie	Biology	1	22

After 2NF: Prescription (selected columns)

Student ID (PK, FK)	Prescription no. (PK)	Drug ID	Doctor ID	Date
S01	1	D01	DR01	2026-09-01
S01	2	D02	DR01	2026-09-03
S02	1	D01	DR02	2026-09-04
S03	1	D02	DR02	2026-09-05

Student details move to Student, where student ID is the primary key. Prescription keeps student ID as a foreign key. Each student fact is stored once, so changing Alex's name requires one update. We can also add a student without adding a prescription.

Drug and doctor descriptions still remain in the intermediate Prescription table, although they are not shown above. They depend on the identified prescription through its drug and doctor. We address these transitive dependencies in 3NF. Dose and quantity stay in Prescription because they describe the event, rather than the student or drug in general.

5 3NF

The intermediate Prescription table contains drug and doctor names. A prescription determines its drug ID, which determines the drug name. Similarly, doctor ID determines doctor name. The highlighted descriptions therefore depend on non-key attributes.

Before 3NF

Student ID	Prescription no.	Drug ID	Drug name	Doctor ID	Doctor name
S01	1	D01	StudyTab A	DR01	Dr Lee
S01	2	D02	StudyTab B	DR01	Dr Lee
S02	1	D01	StudyTab A	DR02	Dr Noor
S03	1	D02	StudyTab B	DR02	Dr Noor

The primary key is still (student ID, prescription number).

Dependencies causing the violations:

- (student ID, prescription number) $\rightarrow$ drug ID $\rightarrow$ drug name, producer.
- (student ID, prescription number) $\rightarrow$ doctor ID $\rightarrow$ doctor name, role, department, age.

After 3NF: StudyDrug

Drug ID (PK)	Drug name	Producer
D01	StudyTab A	Producer A
D02	StudyTab B	Producer B

After 3NF: Doctor

Doctor ID (PK)	Name	Role	Department	Age
DR01	Dr Lee	Physician	Student Health	40
DR02	Dr Noor	Physician	General Practice	45

After 3NF: Prescription

Prescription ID (PK)	Student ID (FK)	Drug ID (FK)	Doctor ID (FK)	Date	Dose (mg)	Tablets
P001	S01	D01	DR01	2026-09-01	5	20
P002	S01	D02	DR01	2026-09-03	10	15
P003	S02	D01	DR02	2026-09-04	5	10
P004	S03	D02	DR02	2026-09-05	10	20

Student remains as shown after 2NF. Drug and doctor details move to their own tables. Prescription keeps their IDs and the event details. We also replace the temporary student-local key with a global prescription ID: (S01, 1) becomes P001, (S01, 2) becomes P002, and so on. This is a key-design choice, separate from removing the transitive dependencies; using a new ID alone would not fix them.

Changing a doctor's name now requires one update. Deleting a prescription does not delete the student, doctor or drug. Foreign keys keep the links valid. Repeated IDs in Prescription are expected because different events can refer to the same person or product.